CampusConnect

Critical Bug Fixes, Security & UI/UX Refinement — Master Prompt

Give this prompt to Gemini / Antigravity with access to the existing CampusConnect repository. Fix the existing project rather than rebuilding it.

Role and working rules

Act as a senior full-stack engineer, security engineer and UI/UX designer. Four first-semester students work on one laptop directly on main. Keep code simple, readable and commented. Preserve the current stack and all working features. Inspect root causes before editing; do not over-engineer or silently remove functionality.

1. CRITICAL — Staff authorization and department isolation

Problem: Cleanliness staff can see other departments' tickets, technician assignments and escalations. This is a backend authorization defect, not merely a UI filtering issue.

- Restrict plumbing, electricity, cleanliness and internet staff to their authorized category/department, and sub-department if applicable. Supervisors see only authorized escalations; admins may see all; students see only their own tickets.
- Inspect authentication, staff profiles, role/department mapping, dashboard queries, API routes and Supabase RLS. Enforce permissions in backend and database, not only client filters.
- Apply restrictions to tickets, history, duplicate reviews, technician assignments, escalations and realtime subscriptions. Block direct URL/API access and role/department self-escalation.
- Tests: cleanliness → plumbing/electricity access denied; plumbing → authorized tickets allowed; student → other student ticket denied; admin → all allowed; manipulated ticket ID denied.

2. Complete Hindi localization

Problem: Switching to Hindi translates only parts of the interface.

- Audit login/registration, student/staff/admin dashboards, complaint intake, suggestions, hostel/floor selectors, ticket tracking, technician assignment, duplicates, SLA, heatmaps, CampusShield, PrivacyGuard, dialogs, toasts, errors, empty/loading states, tooltips and Demo Clock Simulation.
- Move all application-controlled strings to locales/en.json and locales/hi.json; use existing LanguageProvider/useT and persist selection across refresh and navigation. Translate dynamic statuses and labels.
- AI follow-up questions should use selected language. Keep user-written text, IDs, email addresses and technical identifiers unchanged. Prefer natural Hindi; do not leave untranslated UI strings.
- Test Hindi across all pages, submission, history, validation errors, SLA modal and refresh.

3. Unique and persistent ticket IDs

Every genuine ticket must receive a unique CC-0001, CC-0002, … identifier.

- Use an atomic PostgreSQL sequence/counter and UNIQUE constraint; never frontend counters, localStorage, timestamps alone or MAX(id)+1.
- Protect concurrent submissions, retries and idempotency; keep IDs stable across restart and devices; preserve existing IDs.
- Possible duplicates are separate from ID collisions. Preserve staff-only Merge/Not Duplicate, reporter counts, history and audit trail; never auto-merge.
- Test sequential and concurrent ticket creation, refresh, restart, second-device login, and private ticket access restrictions.

4. Cross-device login and persistent accounts

Problem: Accounts registered on one laptop cannot reliably sign in on another. Diagnose the actual cause before replacing anything.

- Inspect Supabase Auth integration, environment/project mismatch, email verification, redirect URLs, session refresh, profile creation, RLS and hidden UI errors. Migrate local-only/mock authentication to Supabase Auth if necessary.
- Students: email/password, email verification, persistent profile, student role. Staff: verified account, authorized admin approval, protected department. Admin: protected role; no public admin self-registration.
- Store credentials using Supabase Auth, never plaintext passwords in custom tables or localStorage. Link profiles to auth user IDs; protect routes/API; do not trust user-selected roles.
- Test register on laptop A → verify → logout → sign in on laptop B → same profile and tickets; refresh persistence; bad password; unverified user; student/staff/admin access boundaries.

5. Minimal, professional UI/UX

Refine existing components, not a complete redesign. Aim for human-centered, production-quality clarity rather than a flashy AI-generated appearance.

- Use white/off-white, primary blue #2F6FDE, neutral borders, consistent typography, subtle shadows, restrained rounded corners, clear hierarchy and spacing. Avoid gradients, glow, clutter, repetitive icons, excessive badges, generic marketing copy and gratuitous animation.
- Login: clear Student/Staff selector, email/password, validation, show-password, verification feedback. Student: prominent Report Issue, recent tickets, simple statuses/navigation.
- Complaint: three clear sections, suggestions, editable AI text, accessible floor map. Staff: floor accordion, clean ticket rows, filters, urgency, assignment/status controls.
- Blueprint/heatmap: aligned rooms, selected state, simple legend. Tracking: legible timeline, timestamps and department. Preserve theme/language toggles.
- Mobile-first at 375px, no horizontal scrolling, touch targets at least 44px, accessible dark mode contrast.

6. Preserve existing functionality

- Keep Gemini analysis, multilingual/Hinglish text and voice, suggestions/autocomplete, interactive floor maps, room-level location, routing, duplicate review, tracking/history, heatmaps, CampusShield, PrivacyGuard, SLA escalation, demo clock, resolution verification and staff dashboards.
- If any existing feature is broken, report it. Do not delete working features just to pass tests.

7. Database and migration safety

- Inspect schema, relationships, RLS, triggers and existing data first. Use incremental SQL migrations; preserve users, tickets and IDs. Avoid destructive resets or unnecessary DROP statements.
- Document manual Supabase setup. If database access is unavailable, supply SQL and steps; do not claim migrations were applied.

8. Verification and regression tests

- Run TypeScript, lint, production build and available automated tests. Check auth, department isolation, ID uniqueness, complete Hindi UI, 375px responsiveness and student-to-staff workflow.
- Use separate student/staff/admin test identities where possible. Report PASS / FAIL / NOT TESTED honestly; never claim unexecuted tests passed.

9. Implementation priority

P0: (1) Department authorization; (2) cross-device authentication; (3) ticket uniqueness. P1: (4) Hindi localization; (5) minimal UI refinement. P2: (6) full regression and remaining fixes. Keep the app runnable between changes.

10. Required final response

- Root cause for each bug; exact files changed and why; SQL migrations and execution steps; security controls and cross-device auth explanation.
- Summary of visual changes; test results (passed/failed/not tested); required environment/manual setup; unresolved issues.

Non-negotiable final instruction

Fix underlying database queries, RLS, authorization, authentication and ID generation—not just the appearance. Preserve functionality, minimize complexity, and make CampusConnect reliable for a live hackathon demo. Never expose Gemini or Supabase service-role secrets. Back up current working code before significant changes.